

LEI/LSIRC · Sistemas Distribuídos — PL01

Sockets TCP em Java (cliente/servidor Echo)

Unidade curricular	Sistemas Distribuídos (LEI + LSIRC)
Aula	PL01 — Sockets TCP em Java: cliente/servidor Echo
Data	semana de 21/09/2026
Docente	Ricardo Costa — rcosta@estg.ipp.pt

Objetivos desta aula

- Identificar os papéis do modelo cliente-servidor com TCP e as classes Java `ServerSocket` e `Socket`.
- Compilar e executar um servidor e um cliente Echo em `localhost` (e, se a rede permitir, entre duas máquinas).
- Modificar um protocolo de texto simples (respostas em maiúsculas, comando BYE).
- Reconhecer e interpretar `ConnectException` e `BindException`, e fechar recursos com `try-with-resources`.
- Observar que um servidor sequencial só atende um cliente de cada vez.

Pré-requisitos

- JDK 17 ou superior instalado, com `java -version` e `javac -version` a funcionar no terminal.
- Um editor de texto ou IDE à escolha (não é exigida nenhuma IDE específica).
- Terminal (Prompt de Comandos/PowerShell no Windows, Terminal no macOS/Linux).

1. Enquadramento

TCP (*Transmission Control Protocol*) é um protocolo de transporte **orientado à ligação** (*connection-oriented*): antes de trocar dados, os dois extremos estabelecem uma ligação, e o TCP garante entrega fiável e ordenada dos bytes enviados. Um **socket** é o extremo dessa ligação; uma ligação TCP fica identificada pelos dois pares IP:porto envolvidos (origem e destino). No modelo cliente-servidor, o servidor fica à escuta num porto conhecido e o cliente inicia a ligação.

Em Java:

- **ServerSocket** — associa-se a um porto e aceita ligações (`accept()`).
- **Socket** — liga-se a um endereço:porto (o cliente) ou representa a ligação aceite (o servidor, por cada cliente).
- Cada **Socket** dá acesso a um fluxo de entrada e a um fluxo de saída para ler e escrever na ligação.

Sequência típica:

Servidor:	Cliente:
1. <code>new ServerSocket(porto)</code>	1. <code>new Socket(host, porto)</code>
2. <code>serverSocket.accept()</code> <- bloqueia até chegar um cliente	
3. ler do cliente / escrever para o cliente	2. escrever para o servidor
4. fechar a ligação	3. ler a resposta
(voltar a 2 para o cliente seguinte)	4. fechar a ligação

`accept()` é **bloqueante**: o servidor fica parado nessa linha até chegar um pedido de ligação. Enquanto está a atender um cliente (dentro do ciclo de leitura/escrita), não volta a chamar `accept()` — por isso um servidor sequencial só atende um cliente de cada vez (ver passo 6).

Todo o código fornecido e as soluções usam **try-with-resources** para o `ServerSocket`, o `Socket` e os fluxos associados: fecham-se sempre sozinhos, mesmo que ocorra um erro, sem precisar de blocos `finally`.

2. Passo: preparar o ambiente

1. Abra um terminal e confirme a versão do JDK:

```
java -version
javac -version
```

Deve aparecer 17 ou superior (por exemplo 21). Se o comando não for reconhecido, o JDK não está no PATH:

- **Windows:** confirme que a variável de ambiente PATH inclui a pasta bin do JDK (Definições → Variáveis de Ambiente). Para verificar rapidamente: `where java` (cmd) ou `$env:Path` (PowerShell).
- **macOS/Linux:** confirme com `echo $PATH` e, se necessário, ajuste o `.zshrc/.bashrc`.

2. Crie uma pasta de trabalho para esta aula e copie para lá os ficheiros `EchoServer.java` e `EchoClient.java` fornecidos.

Sobre o porto usado nesta aula: todo o código usa o porto 5050 por omissão. Se esse porto já estiver ocupado por outro programa na sua máquina (o servidor mostra uma `BindException` logo ao arrancar), tanto o servidor como o cliente aceitam um porto alternativo como argumento — por exemplo `java EchoServer 5051` e `java EchoClient localhost 5051` — use o mesmo porto dos dois lados.

3. Passo: servidor Echo fornecido

Ficheiro `EchoServer.java` (fornecido). Aceita ligações num porto (5050 por omissão) e devolve a cada cliente, linha a linha, exatamente o que recebe.

Compile (a opção `-encoding UTF-8` garante que os acentos do código-fonte são lidos corretamente em qualquer sistema operativo):

```
javac --release 17 -encoding UTF-8 EchoServer.java
```

Corra (porto por omissão, 5050):

```
java EchoServer
```

Checkpoint — deve aparecer no terminal e ficar parado à espera (o processo não termina sozinho):

Servidor Echo à escuta no porto 5050...

Se não aparecer isto, ou se o terminal devolver logo o prompt, algo falhou na compilação ou no arranque — reveja a mensagem de erro antes de continuar. Deixe este terminal aberto: o servidor tem de continuar a correr para os passos seguintes.

Se aparecer um aviso da firewall do sistema operativo (Windows Defender ou equivalente) a perguntar se o Java pode aceitar ligações de rede, pode permitir; em `localhost` o servidor funciona mesmo que recuse.

Pergunta para verificar que percebeu o código: que recursos fecha o `try-with-resources` da linha do `ServerSocket`, e o de dentro do ciclo?

4. Passo: cliente Echo fornecido

Ficheiro `EchoClient.java` (fornecido). Recebe o host e o porto como argumentos opcionais (por omissão `localhost` e 5050).

Num **segundo terminal**, compile:

```
javac --release 17 -encoding UTF-8 EchoClient.java
```

Corra e escreva algumas linhas, terminando com Ctrl+D (Linux/macOS) ou Ctrl+Z seguido de Enter (Windows):

```
java EchoClient
```

Checkpoint — escrevendo `ola mundo` e depois como `estas`, a saída do cliente é:

```
Ligado a localhost:5050. Sair: Ctrl+D (Linux/macOS) ou Ctrl+Z+Enter (Windows).
eco: ola mundo
eco: como estas
```

E no terminal do servidor deve aparecer, pela mesma ordem:

```
Cliente ligado: /127.0.0.1:<porto-efêmero>
Recebido: ola mundo
Recebido: como estas
Cliente desligou: /127.0.0.1:<porto-efêmero>
```

(<porto-efêmero> é um número diferente a cada ligação — é o porto de origem escolhido pelo sistema operativo para o cliente; não precisa de decorar o valor.)

Nota sobre acentos em Windows: a consola do Windows (cp850/cp1252) pode não mostrar corretamente palavras com acentos escritas por si durante a sessão (ex. “olá”), mesmo com o código a compilar e a correr bem. Nos testes desta aula, escreva frases sem acentuação (como no exemplo acima) para evitar esta confusão; o motivo (codificação de caracteres) discute-se em aula.

5. Passo: provocar os erros

5.1 Cliente sem servidor à escuta

Feche o servidor (Ctrl+C no terminal onde está a correr) e, no terminal do cliente, corra outra vez:

```
java EchoClient
```

Checkpoint — mensagem (o texto entre parênteses vem diretamente do sistema operativo e varia consoante o SO):

```
Não foi possível ligar a localhost:5050 - o servidor está em baixo? (Connection refused)
```

Em Linux/macOS o texto entre parênteses é `Connection refused`; em Windows costuma aparecer `Connection refused: connect`. Em qualquer dos casos corresponde a uma `java.net.ConnectException` apanhada pelo cliente: não há ninguém à escuta naquele endereço:porto. Esta é também uma instância concreta da falácia «a rede é fiável» (T01) — a ligação pode simplesmente não existir do outro lado.

5.2 Dois servidores no mesmo porto

Arranque o servidor outra vez num terminal (`java EchoServer`) e, **noutro terminal**, tente arrancar um segundo:

```
java EchoServer
```

Checkpoint — mensagem (o texto depois de `BindException`: varia consoante o SO):

```
Não foi possível iniciar o servidor no porto 5050: java.net.BindException: Address already in use
```

Em Linux/macOS o texto é `Address already in use`; em Windows costuma aparecer `Address already in use: bind`. Isto é uma `java.net.BindException`: o porto já está ocupado pelo primeiro servidor. Só um processo pode escutar num dado porto de cada vez. Feche o segundo terminal; deixe o primeiro servidor a correr para o passo seguinte.

6. Passo: dois clientes em simultâneo

Com o servidor ainda a correr, abra **dois terminais de cliente**. No primeiro, corra `java EchoClient` e escreva uma linha (não feche o cliente). No segundo terminal, corra também `java EchoClient` e escreva uma linha.

Observação esperada: o segundo cliente mostra a mensagem de ligação (`Ligado a localhost:5050...`), mas **não recebe eco** enquanto o primeiro cliente estiver ligado — fica parado à espera. Só depois de

terminar o primeiro cliente (Ctrl+D no Linux/macOS, Ctrl+Z e Enter no Windows) é que o segundo recebe a resposta à linha que escreveu.

Pergunta de reflexão (sem resposta neste guia — discussão em aula): porque é que o segundo cliente consegue “ligar-se” mas não recebe resposta enquanto o primeiro está ativo? Que linha do código do servidor é responsável por este comportamento?

7. Exercício 1 — protocolo em maiúsculas + BYE

Modifique (ou crie a partir do fornecido) um servidor e um cliente Echo com este protocolo:

- O servidor devolve cada linha recebida **em maiúsculas**.
- Quando recebe a linha BYE (sem distinguir maiúsculas/minúsculas), o servidor responde BYE e fecha a ligação com aquele cliente — mas continua pronto para aceitar o próximo.
- O cliente termina de forma limpa (sem exceções) assim que envia BYE e recebe a confirmação.

Checkpoint — o essencial a confirmar (os textos exatos das suas mensagens podem ser diferentes dos da solução do docente; o que importa é o comportamento):

- Ao escrever `ola mundo`, o cliente mostra a resposta do servidor em maiúsculas: `OLA MUNDO`.
 - Ao escrever BYE, o cliente recebe a confirmação do servidor e **termina sem exceção nem stack trace**.
 - Do lado do servidor, a ligação com aquele cliente fecha-se, mas o processo do servidor continua a correr.
 - Se ligar um novo cliente a seguir, o servidor aceita-o sem ser preciso reiniciar (mesmo terminal, mesmo processo).
-

8. Exercício 2 — entre duas máquinas (se a rede permitir)

Com um colega, um de vocês corre o servidor e o outro o cliente, em máquinas diferentes na mesma rede.

1. Descubra o endereço IP da máquina que vai correr o servidor:
 - **Windows:** `ipconfig` (ver “Endereço IPv4”)
 - **Linux:** `ip addr` (ver a interface de rede ativa, ex. `wlan0/eth0`)
 - **macOS:** `ifconfig` (ver a interface ativa, ex. `en0`)
2. No terminal do servidor: `java EchoServer` (porto por omissão, 5050).
3. No terminal do cliente, na outra máquina: `java EchoClient <IP-do-servidor> 5050`.

Se a rede/firewall bloquear a ligação (muito provável em eduroam ou com firewall pessoal ativa): confirme que ambos estão na mesma rede local e que a firewall da máquina do servidor permite ligações de entrada ao Java na porta escolhida; se ainda assim não funcionar, **fica cumprido o objetivo correndo os dois papéis (servidor e cliente) em dois terminais da mesma máquina**, como nos passos 3–4.

9. Exercício 3 — servidor de comandos

Escreva um servidor que interpreta comandos de texto, um por linha:

- `TIME` → devolve a hora atual do servidor.
- `ADD a b` → devolve a soma de `a` e `b` (inteiros).
- qualquer outro comando (ou mal formado) → devolve `ERRO`.

O servidor deve continuar a aceitar um novo cliente depois de o anterior se desligar (ciclo de `accept`). Pare primeiro o `EchoServer` do passo 3 (ou use outro porto) antes de arrancar este servidor de comandos; pode continuar a usar o `EchoClient` fornecido como cliente, já que ele só envia linhas e mostra a resposta.

Checkpoint — enviando `TIME`, depois `ADD 2 3`, depois `MULT 2 3` (comando inexistente):

Ligado a `localhost:5050`. Sair: Ctrl+D (Linux/macOS) ou Ctrl+Z+Enter (Windows).
eco: 15:14:30

eco: 5
eco: ERRO

(A hora devolvida por **TIME** reflete o momento em que correu o teste — o formato é **HH:mm:ss**.)

10. Para saber mais

- Oracle, *The Java Tutorials — All About Sockets*: <https://docs.oracle.com/javase/tutorial/networking/sockets/>
- Javadoc (Java 17): `java.net.Socket` e `java.net.ServerSocket`
- Tanenbaum & Van Steen, *Distributed Systems*, 4.^a ed., cap. 4 (*Communication*)